



MIDDLE EAST TECHNICAL UNIVERSITY

MMI714

Generative Models for Multimedia

Normalizing Flows

Welcome

This is MMI714

“Generative Models for Multimedia”

This week we will talk about:

Normalizing Flows,
Density Estimation, Invertible Neural
Networks, Change of Variables,
Coupling Flows





Probabilistic Models

$$P(A|B) = \frac{\text{Probability of } A \text{ and } B}{\text{Probability of } B}$$

$P(A \cap B)$
 Probability of A given B

Machine Learning by Probabilistic Models:

- Examples: Naive Bayes, Logistic Regression, Hidden Markov Models (HMMs), Gaussian Mixture Models (GMMs).
- These models focus on estimating the conditional probability distributions of the target variables given the input variables.
- They make assumptions about the functional form of the underlying distributions.

About METU	Academia	Student Affairs	Services	Campus Life	Research	Prospective Students
Academic Catalog						
Departments Reporting to President's Office	Main >> Cognitive Sciences > Courses					
Faculty of Architecture						
Faculty of Arts and Sciences						
Faculty of Economic and Administrative Sciences						
Faculty of Education						
Faculty of Engineering						
Graduate School of Applied Mathematics						
Graduate School of Informatics						
Graduate School of Marine Sciences						
Graduate School of Natural and Applied Sciences						
Graduate School of Social Sciences						
School of Foreign Languages						
[NCC] NCC Engineering Programs						
[NCC] NCC Graduate Programs						
[NCC] NCC Social Sciences Programs						
[NCC] School of Foreign Languages						
COGS566 PROBABILISTIC MODELS OF COGNITION						
Course Code:		9020566				
METU Credit (Theoretical-Laboratory hours/week):		3(0-0)				
ECTS Credit:		8.0				
Department:		Cognitive Sciences				
Language of Instruction:		English				
Level of Study:		Graduate				
Course Coordinator:						
Offered Semester:		Fall Semesters.				
Course Objectives						
The course aims to show how probabilistic and Bayesian models can be applied for modelling different aspects of cognition including learning, causal reasoning and decision making. We will use probabilistic programming languages to apply these models.						
Course Content						

Probabilistic Generative Models

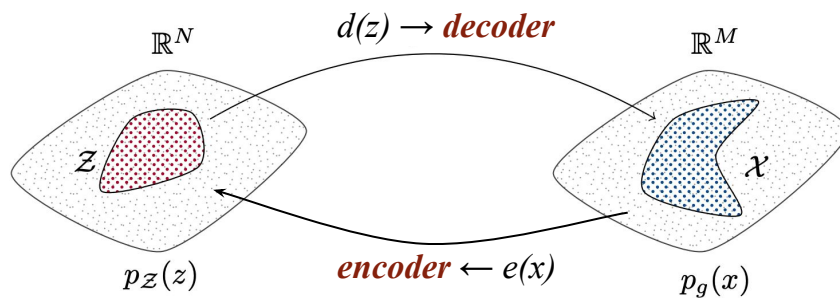
- Examples: Gaussian Mixture Models (GMMs), Variational Autoencoders (VAEs), **Normalizing Flows**, Generative Adversarial Networks (GANs).
- These models aim to learn the joint probability distribution of both the input (z) and target (x) variables.
- They capture the underlying data generation process and can generate new samples.

Probabilistic Generative Models

- Examples: Gaussian Mixture Models (GMMs), **Variational Autoencoders (VAEs)**, Normalizing Flows, Generative Adversarial Networks (GANs).

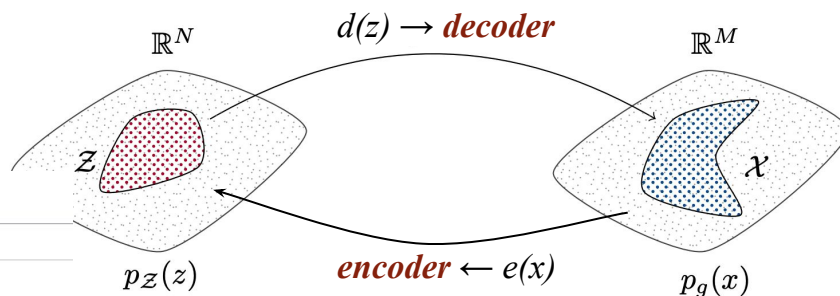
? These models aim to learn the joint probability distribution of both the input (z) and target (x) variables.

- They capture the underlying data generation process and can generate new samples.



Probabilistic Generative Models (VAE)

- In the case of VAEs, although the focus is primarily on modeling the conditional distributions $p(z|x)$ and $p(x|z)$, the joint distribution $p(x, z)$ can still be inferred through the training process.
 - The encoder part of the VAE approximates the posterior distribution $p(z|x)$, while the decoder approximates the conditional distribution $p(x|z)$.
 - VAEs aim to learn the joint distribution as closely as possible.



Evidence lower bound

[Article](#) [Talk](#)

From Wikipedia, the free encyclopedia

In [variational Bayesian methods](#), the **evidence lower bound** (often abbreviated **ELBO**, also sometimes called the **variational lower bound**^[1] or **negative variational free energy**) is a useful lower bound on the log-likelihood of some observed data.

Probabilistic Generative Models

- These models aim to learn the joint probability distribution of both the input (z) and target (x) variables.

?

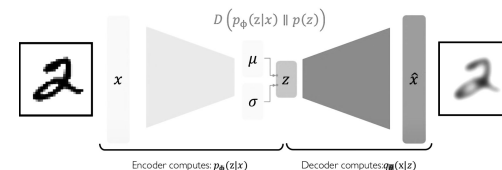
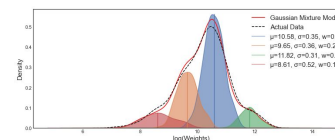
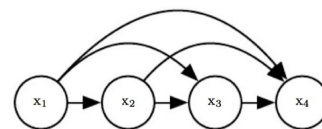
Are there any conventional machine learning probabilistic models that aim at finding the joint probability distribution?

- Yes, there are conventional machine learning probabilistic models that aim to estimate the joint probability distribution. One such example is the Gaussian Mixture Model (GMM). But, for the same reason, it is also a generative model!
- (and that's why they do not have a latent space)

Probabilistic Generative Models (so far)

Modelling Approach:

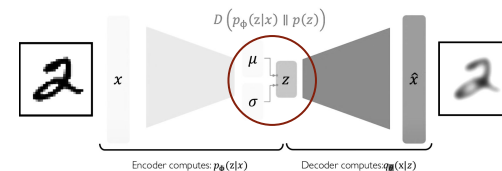
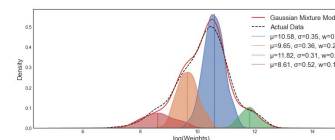
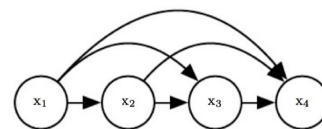
- Autoregressive Models: *capture the dependencies within a sequence of variables by modeling the conditional distribution of each variable given its previous values.*
- GMMs: *model the joint distribution of the observed variables by estimating the means, covariances, and mixture weights of the Gaussian components.*
- VAEs: *1) combine a probabilistic encoder and decoder to learn a low-dimensional representation (latent space) of the data, 2) and model the joint distribution of the latent variables and the observed data by approximating the posterior distribution using variational inference.*



Probabilistic Generative Models (so far)

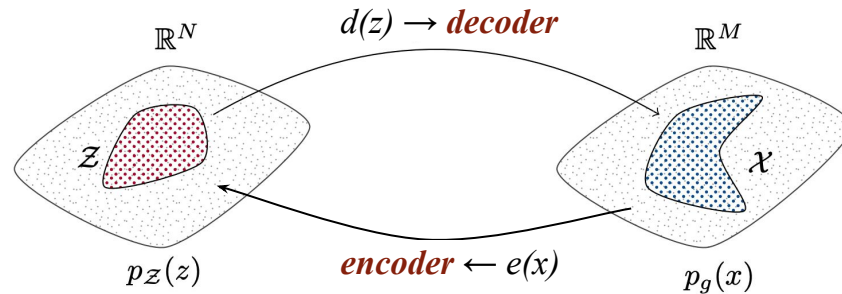
Modelling Approach:

- Autoregressive Models: *capture the dependencies within a sequence of variables by modeling the conditional distribution of each variable given its previous values.*
- GMMs: *model the joint distribution of the observed variables by estimating the means, covariances, and mixture weights of the Gaussian components.*
- VAEs: *1) combine a probabilistic encoder and decoder to learn a low-dimensional representation (**latent space**) of the data, 2) and model the joint distribution of the latent variables and the observed data by approximating the posterior distribution using variational inference.*



Probabilistic Generative Models

- In the case of VAEs, the encoder and decoder are designed to function inverse operations, but this is an approximation.
- What if there was just one function, say “theCoder”, that we could take the inverse of?

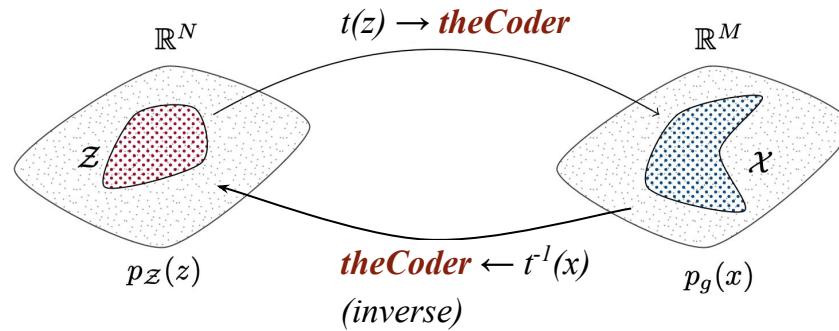




The Coder

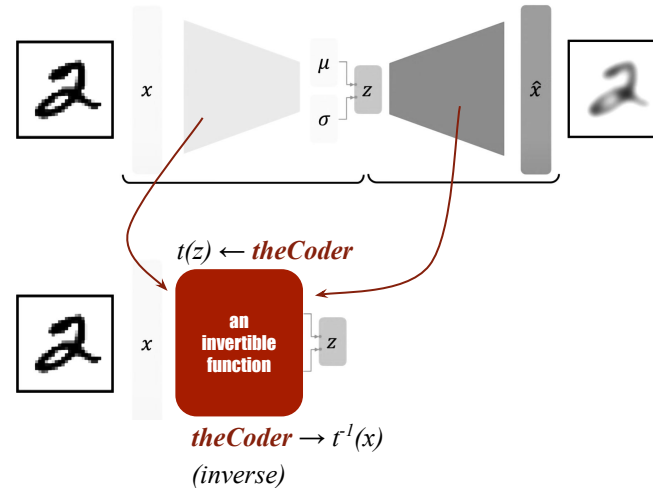
Probabilistic Generative Models

- In the case of VAEs, the encoder and decoder are designed to function inverse operations, but this is an approximation.
- What if there was just one function, say “theCoder”, that we could take the inverse of?



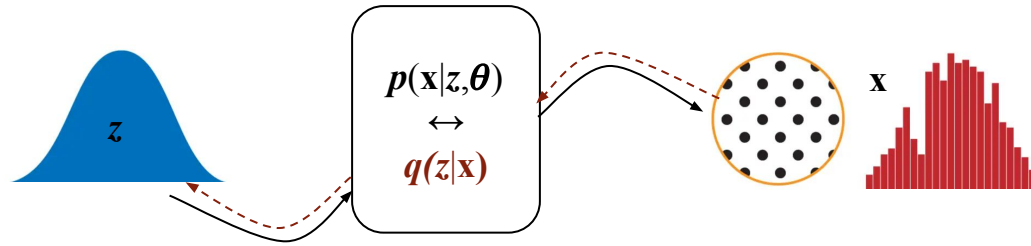
Probabilistic Generative Models

- In that case, the encoder and the decoder would be one, invertible function
 - VAE:
 - Something that we won't have to worry about the decoder estimating the posterior probability incorrectly.

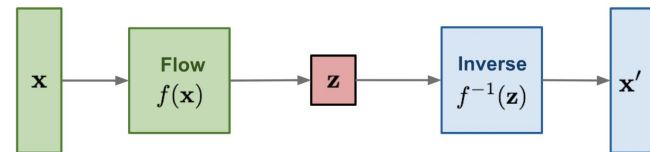


Flow-based Generative Models

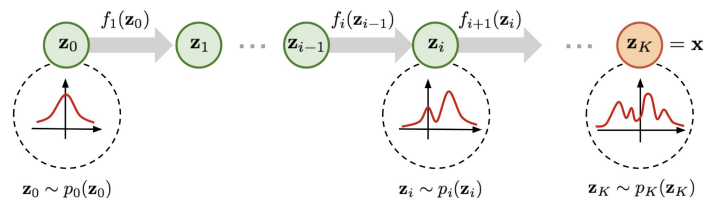
- Flow-based models enable efficient density estimation and generation by sampling from the latent space and applying the inverse transformations.



Flow-based Generative Models



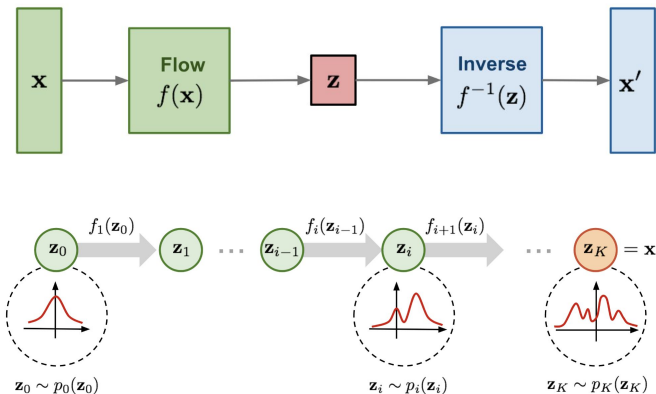
- Flow-based models enable efficient density estimation and generation by sampling from the latent space and applying the inverse transformations.
- The flow is usually constructed by a sequence of invertible transformations.



- Unlike VAEs (and GANs, which we'll later learn), the model explicitly learns the data distribution
 - and therefore the loss function will simply be the negative log-likelihood, (because we have the explicit distribution. We'll talk about this)

Flow-based Generative Models

- built on invertible transformations
 - $f(\cdot)$ has to be invertible
 - $f(\cdot)$ has to be differentiable
- efficient
 - efficient to sample: just use $f(\mathbf{x})$
 - efficient to evaluate: just use $f^{-1}(\mathbf{z})$
- continuous latent space $\mathbf{z}$
- straightforward to train with likelihood
- So what is $f(\cdot)$?





Flow-based GMs Literature

COMMUN. MATH. SCI.
Vol. 8, No. 1, pp. 217–233

© 2010 International Press

DENSITY ESTIMATION BY DUAL ASCENT OF THE LOG-LIKELIHOOD*

ESTEBAN G. TABAK[†] AND ERIC VANDEN-ELJNDEN[‡]

Dedicated to the sixtieth birthday of Professor Andrew Majda

A methodology is developed to assign, from an observed sample, a joint-probability distribution to a set of continuous variables.

"Normalizing flow" was introduced first time as a term.

$$\rho(x) = \begin{cases} e^{-(x+1)} & \text{if } x \geq -1, \\ 0 & \text{otherwise.} \end{cases} \quad (7.1)$$

The top-left plot displays a histogram of the sample. The top row shows the evolution of this histogram, as the sample is deformed by the **normalizing flow**. The bottom row of plots shows the evolution of the corresponding estimate of the original density, computed on a uniform grid.

Normalizing: In probability theory, normalizing refers to the process of ensuring that a probability density function integrates to 1 over its entire domain. In other words, the integral of the probability density function over all possible values of the random variable should be equal to 1.

Flow: In the context of transformations, a "flow" refers to the idea of smoothly transforming data points from one distribution to another using a sequence of transformations. The term "flow" implies a continuous, gradual change, like the flow of water in a river.

Introduces the idea - 2010

Flow-based GMs Literature

COMMUN. MATH. SCI.
Vol. 8, No. 1, pp. 217–233

© 2010 International Press

DENSITY ESTIMATION BY DUAL ASCENT OF THE LOG-LIKELIHOOD*

ESTEBAN G. TABAK[†] AND ERIC VANDEN-EIJNDEN[‡]

Dedicated to the sixtieth birthday of Professor Andrew Majda

Introduces the idea - 2010

218

A DUAL ASCENT OF THE LOG-LIKELIHOOD

pursuit [5], which seeks optimal directions for functional fitting. Within this latter framework, the Gaussianization procedure proposed in [6] has some commonality with the methodology developed here.

We propose to perform density estimation by mapping the x 's into a new set of variables y with known probability density $\mu(y)$. Then the density $\rho(x)$ is given by

$$\rho(x) = J_y(x) \mu(y(x)), \quad (1.1)$$

where $J_y(x)$, the Jacobian of the map $y(x)$, is computed explicitly alongside the map. The map $y(x)$ is built as an infinite composition of infinitesimal transformations, i.e., by introducing a flow $z = \phi_t(x)$ such that

$$\phi_0(x) = x, \quad \lim_{t \rightarrow \infty} \phi_t(x) = y(x). \quad (1.2)$$

Associated with the map $\phi_t(x)$ we can introduce the density $\tilde{\rho}_t(x)_t$ given by (1.1) but with $y(x)$ replaced by $\phi_t(x)$:

$$\tilde{\rho}_t(x) = J_{\phi_t}(x) \mu(\phi_t(x)). \quad (1.3)$$

If (1.2) holds, then from (1.1) the density $\tilde{\rho}_t(x)$ satisfies

$$\tilde{\rho}_0(x) = \mu(x) \quad \lim_{t \rightarrow \infty} \tilde{\rho}_t(x) = \rho(x).$$

Flow-based GMs Literature

COMMUN. MATH. SCI.
Vol. 8, No. 1, pp. 217–233

© 2010 International Press

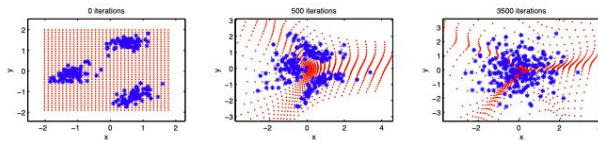
DENSITY ESTIMATION BY DUAL ASCENT OF THE LOG-LIKELIHOOD*

ESTEBAN G. TABAK[†] AND ERIC VANDEN-EIJNDEN[‡]

Dedicated to the sixtieth birthday of Professor Andrew Majda

A methodology is developed to assign, from an observed sample, a joint-probability distribution to a set of continuous variables.

“Normalizing flow” was introduced first time as a term.



The paper can be considered the pioneering work for normalizing flows as it introduced the fundamental idea of using transformations to estimate complex probability distributions. The paper proposed an algorithm that iteratively mapped an observed sample of continuous variables onto a jointly-Gaussian set, aiming to maximize the log-likelihood of the observations. This iterative process allowed the sample's empirical distribution to evolve from a simple distribution, such as exponential, towards the target Gaussian distribution. In doing so, the paper laid the groundwork for the concept of normalizing flows.

Introduces the idea - 2010



Flow-based GMs Literature

COMMUN. MATH. SCI.
Vol. 8, No. 1, pp. 217–233

© 2010 International Press

DENSITY ESTIMATION BY DUAL ASCENT OF THE LOG-LIKELIHOOD*

ESTEBAN G. TABAK[†] AND ERIC VANDEN-EIJNDEN[‡]

Dedicated to the sixtieth birthday of Professor Andrew Majda

232

A DUAL ASCENT OF THE LOG-LIKELIHOOD

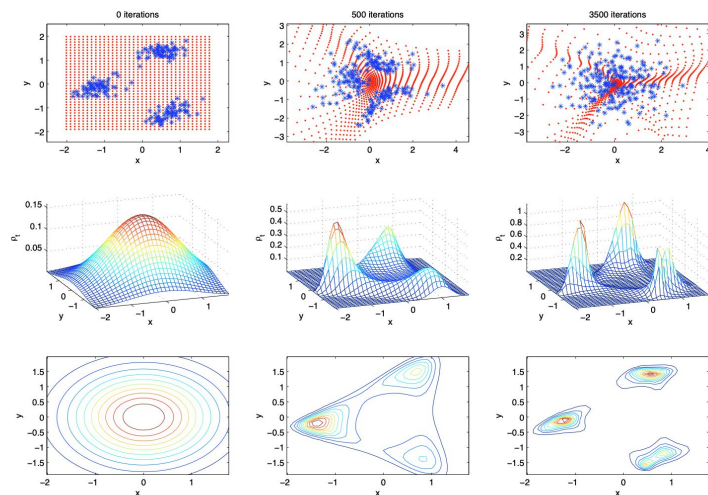


FIG. 7.4. Illustration of the procedure on a two-dimensional example with a density made by mixture of 3 Gaussians. The top panels show the evolution of the 200 sample points as well as that of the grid carried passively by the algorithm. The middle and bottom panels show, respectively, the three-dimensional plot and contourplots of the estimated density as it evolves from a Gaussian toward the estimation of the Gaussian mixture associated with the sample.

Flow-based GMs Literature

A Family of Nonparametric Density Estimation Algorithms

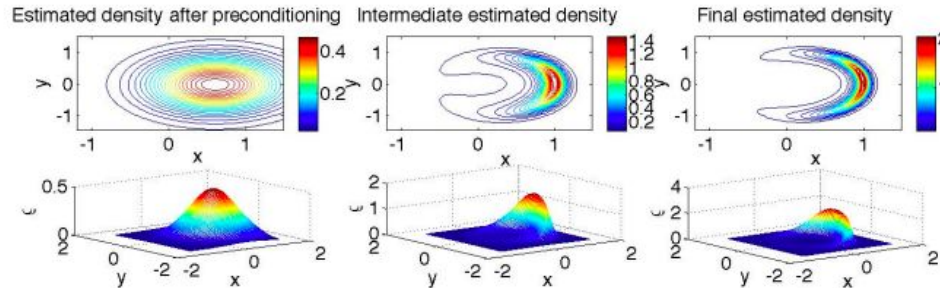
E. G. TABAK
Courant Institute

CRISTINA V. TURNER
FaMAF, Universidad Nacional de Córdoba

Built on the one [Tabak and Vanden-Eijnden 2010], generalizes the idea and provides solutions for efficient convergence.

Introduces the idea - 2010

Generalizes the idea - 2011



Based on Tabak 2010, Tabak 2011 generalizes the idea by shifting towards non-parametric techniques and expands the method's flexibility and adaptability by eschewing (avoiding) specific distributional assumptions.



Flow-based GMs Literature

High-Dimensional Probability Estimation with Deep Density Models

Oren Rippel*
Massachusetts Institute of Technology,
Harvard University
rippel@math.mit.edu

Ryan Prescott Adams†
Harvard University
rpa@seas.harvard.edu

Uses deep learning techniques to construct a one-to-one and onto mapping from the original data space to a representation space.

Introduces the idea - 2010

Generalizes the idea - 2011

Application to DL - 2013

Abstract

One of the fundamental problems in machine learning is the estimation of a probability distribution from data. Many techniques have been proposed to study the structure of data, most often building around the assumption that observations lie on a lower-dimensional manifold of high probability. It has been more difficult, however, to exploit this insight to build explicit, tractable density models for

The assumption is that the data points in a high-dimensional space actually lie on or close to a lower-dimensional manifold (a curved subspace) embedded within the higher-dimensional space. Additionally, this manifold is assumed to be associated with high probability regions of the data distribution.



Flow-based GMs Literature

High-Dimensional Probability Estimation with Deep Density Models

Oren Rippel*
Massachusetts Institute of Technology,
Harvard University
rippel@math.mit.edu

Ryan Prescott Adams†
Harvard University
rpa@seas.harvard.edu

Uses deep learning techniques to construct a one-to-one and onto mapping from the original data space to a representation space.

Introduces the idea - 2010

Generalizes the idea - 2011

Application to DL - 2013

Abstract

One of the fundamental problems in machine learning is the estimation of a probability distribution from data. Many techniques have been proposed to study the structure of data, most often building around the assumption that observations lie on a lower-dimensional manifold of high probability. It has been more difficult, however, to exploit this insight to build explicit, tractable density models for high-dimensional data. In this paper, we introduce the *deep density model* (DDM), a new approach to density estimation. We exploit insights from deep learning to construct a bijective map to a rep-

The challenge of using the assumption of a lower-dimensional manifold of high probability to construct explicit and tractable density models for high-dimensional data.

Flow-based GMs Literature

Introduces the idea - 2010

Generalizes the idea - 2011

Application to DL - 2013

High-Dimensional Probability Estimation with Deep Density Models

Oren Rippel*
Massachusetts Institute of Technology,
Harvard University
rippel@math.mit.edu

Ryan Prescott Adams†
Harvard University
rpa@seas.harvard.edu

Uses deep learning techniques to construct a one-to-one and onto mapping from the original data space to a representation space.



(a) Generations from a 3-layer DDM on MNIST, with diversified marginals Beta ($\cdot$; 0.015, 0.8).

- *Uses deep learning techniques to construct a one-to-one and onto mapping from the original data space to a representation space.*
- *In this representation space, the data distribution becomes approximately factorized, meaning that the joint distribution can be expressed as a product of individual marginal distributions.*
- *Additionally, each dimension in the representation space has an identical and known marginal density, which facilitates tractable density modeling and enables various probabilistic tasks to be tackled efficiently.*

$$p(x_1, x_2, \dots, x_n) \approx p(x_1) \cdot p(x_2) \cdot \dots \cdot p(x_n)$$

Flow-based GMs Literature

NICE: NON-LINEAR INDEPENDENT COMPONENTS ESTIMATION

Laurent Dinh David Krueger Yoshua Bengio*
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

It is based on the idea that a good representation is one in which the data has a distribution that is easy (independent) to model.

Introduces the idea - 2010

Generalizes the idea - 2011

Application to DL - 2013

Independent Latent Space- 2014

ABSTRACT

We propose a deep learning framework for modeling complex high-dimensional densities called Non-linear Independent Component Estimation (NICE). It is based on the idea that **a good representation is one in which the data has a distribution that is easy to model**. For this purpose, a non-linear deterministic transformation of the data is learned that maps it to a latent space so as to make the transformed data conform to a factorized distribution, i.e., resulting in independent latent variables. We parametrize this transformation so that computing the

In this context, a "good representation" refers to a latent space where the data can be transformed to follow a simple, factorized distribution, ideally with independent components.

Flow-based GMs Literature

NICE: NON-LINEAR INDEPENDENT COMPONENTS ESTIMATION

Laurent Dinh · David Krueger · Yoshua Bengio*
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

It is based on the idea that a good representation is one in which the data has a distribution that is easy to model.

Introduces the idea - 2010

Generalizes the idea - 2011

Application to DL - 2013

Independent Latent Space- 2014

ABSTRACT

We propose a deep learning framework for modeling complex high-dimensional densities called Non-linear Independent Component Estimation (NICE). It is based on the idea that a good representation is one in which the data has a distribution that is easy to model. For this purpose, a non-linear deterministic transformation of the data is learned that maps it to a latent space so as to make the transformed data conform to a factorized distribution, i.e., resulting in independent latent variables. We parametrize this transformation so that computing the

NICE achieves the desired representation by learning a non-linear deterministic transformation that maps the input data to a latent space. The transformation is designed to capture complex non-linear relationships present in the data.

Flow-based GMs Literature

Introduces the idea - 2010

Generalizes the idea - 2011

Application to DL - 2013

Independent Latent Space- 2014

NICE: NON-LINEAR INDEPENDENT COMPONENTS ESTIMATION

Laurent Dinh · David Krueger · Yoshua Bengio*
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

It is based on the idea that a good representation is one in which the data has a distribution that is easy to model.



While DDM (2013) explored the potential of deep density models for high-dimensional probability estimation, NICE (2014) narrows the focus to independent component estimation using non-linear transformations.

Flow-based GMs Literature

NICE: NON-LINEAR INDEPENDENT COMPONENTS ESTIMATION

Laurent Dinh David Krueger Yoshua Bengio*
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

It is based on the idea that a good representation is one in which the data has a distribution that is easy to model.

Introduces the idea - 2010

Generalizes the idea - 2011

Application to DL - 2013

Independent Latent Space- 2014



NICE itself is not classified as a paper solely focused on normalizing flows, it played a significant role in the development of the concepts related to invertible transformations and their applications in density estimation.

Flow-based GMs Literature

Introduces the idea - 2010

Generalizes the idea - 2011

Application to DL - 2013

Independent Latent Space- 2014

NF for Variational Inf. - 2015

Variational Inference with Normalizing Flows

Danilo Jimenez Rezende
Shakir Mohamed
Google DeepMind, London

DANILOR@GOOGLE.COM
SHAKIR@GOOGLE.COM

Shifts the focus to approximate posterior distributions in the context of variational inference.

While both NICE and "Variational Inference with Normalizing Flows" use normalizing flows as a common element, they have different objectives and contexts. NICE focuses on independent component estimation and density modeling, while "Variational Inference with Normalizing Flows" explores using normalizing flows for specifying flexible and scalable approximate posterior distributions (i.e. the distribution of the latent variables given the observed data.) in the context of variational inference.

Flow-based GMs Literature

Introduces the idea - 2010

Generalizes the idea - 2011

Application to DL - 2013

Independent Latent Space- 2014

NF for Variational Inf. - 2015

Variational Inference with Normalizing Flows

Danilo Jimenez Rezende
Shakir Mohamed
Google DeepMind, London

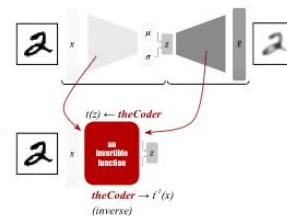
DANILOR@GOOGLE.COM
SHAKIR@GOOGLE.COM

Shifts the focus to approximate posterior distributions in the context of variational inference.

Abstract

The choice of approximate posterior distribution is one of the core problems in variational inference. Most applications of variational inference employ simple families of posterior approximations in order to allow for efficient inference, focusing on mean-field or other simple structured approximations. This restriction has a significant impact on the quality of inferences made using variational methods. We introduce a new approach for specifying flexible, arbitrarily complex and scalable approximate posterior distributions. Our approximations are distributions constructed through a normalizing flow, whereby a simple initial density is transformed into a more complex one by applying a sequence of invertible transformations until a desired level of complexity is attained. We use this view of normalizing flows to develop categories of finite and infinites-

- We propose the specification of approximate posterior distributions using normalizing flows, a tool for constructing complex distributions by transforming a probability density through a series of invertible mappings (sect. 3). Inference with normalizing flows provides a tighter, modified variational lower bound with additional terms that only add terms with linear time complexity (sect 4).
- We show that normalizing flows admit infinitesimal flows that allow us to specify a class of posterior approximations that in the asymptotic regime is able to recover the true posterior distribution, overcoming one oft-quoted limitation of variational inference.
- We present a unified view of related approaches for improved posterior approximation as the application of special types of normalizing flows (sect 5).
- We show experimentally that the use of general normalizing flows systematically outperforms other competing approaches for posterior approximation.



Flow-based GMs Literature

real-valued non-volume preserving (real NVP) transformations, a set of powerful invertible and learnable transformations, resulting in an unsupervised learning algorithm with exact log-likelihood computation

Published as a conference paper at ICLR 2017

DENSITY ESTIMATION USING REAL NVP

Laurent Dinh*

Montreal Institute for Learning Algorithms
University of Montreal
Montreal, QC H3T 1J4

Jascha Sohl-Dickstein
Google Brain

Samy Bengio
Google Brain

Introduces the idea - 2010

Generalizes the idea - 2011

Application to DL - 2013

Independent Latent Space- 2014

NF for Variational Inf. - 2015

Proof of concept - 2016



The approach allows for exact log-likelihood computation, exact sampling, and exact inference of latent variables. It demonstrates promising results in modeling natural images and showcases the advantages of using real NVP transformations in probabilistic modeling tasks.

Flow-based GMs Literature

Glow: Generative Flow with Invertible 1×1 Convolutions

Diederik P. Kingma*, Prafulla Dhariwal*
OpenAI, San Francisco

*On top of the idea of NVP, they trained it harder
and got very realistic results.*

Introduces the idea - 2010

Generalizes the idea - 2011

Application to DL - 2013

Independent Latent Space- 2014

NF for Variational Inf. - 2015

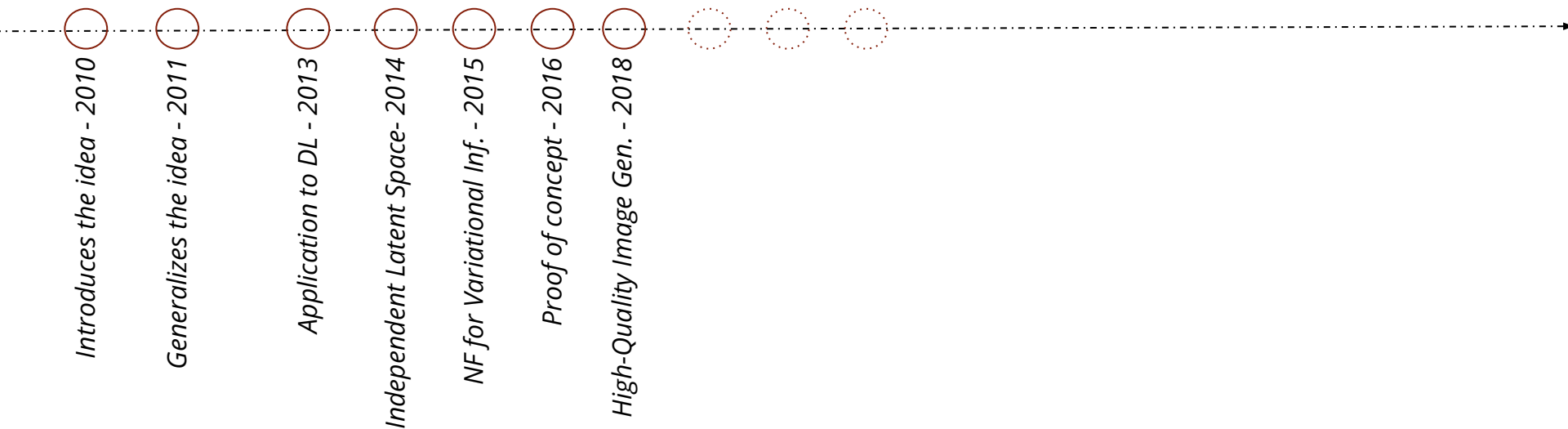
Proof of concept - 2016

High-Quality Image Gen. - 2018



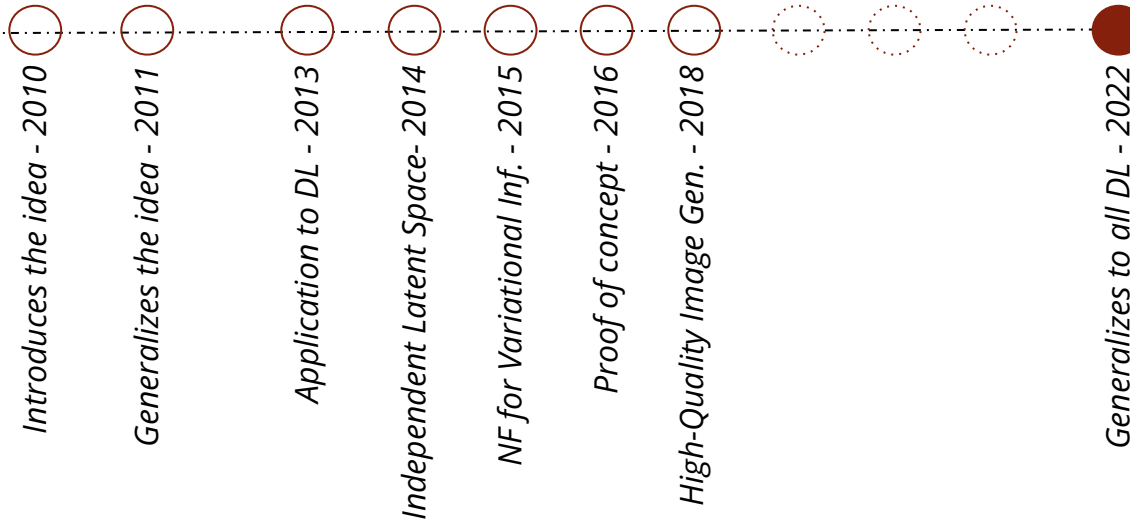
Glow was a key paper in the advancement of normalizing flows. It introduced a powerful architecture that incorporates invertible 1×1 convolutions, leading to improved flexibility, scalability, and high-quality image generation. Glow represented a significant step forward in the development of generative flow models and their applications in various practical scenarios.

Flow-based GMs Literature



Flow-based GMs Literature

argues that by introducing a stochastic inverse pass and monitoring the likelihood contribution, certain neural network architectures can be considered generalized normalizing flows.



Flowification: Everything is a Normalizing Flow

Bálint Máté*
University of Geneva
balint.mate@unige.ch

Samuel Klein*
University of Geneva
samuel.klein@unige.ch

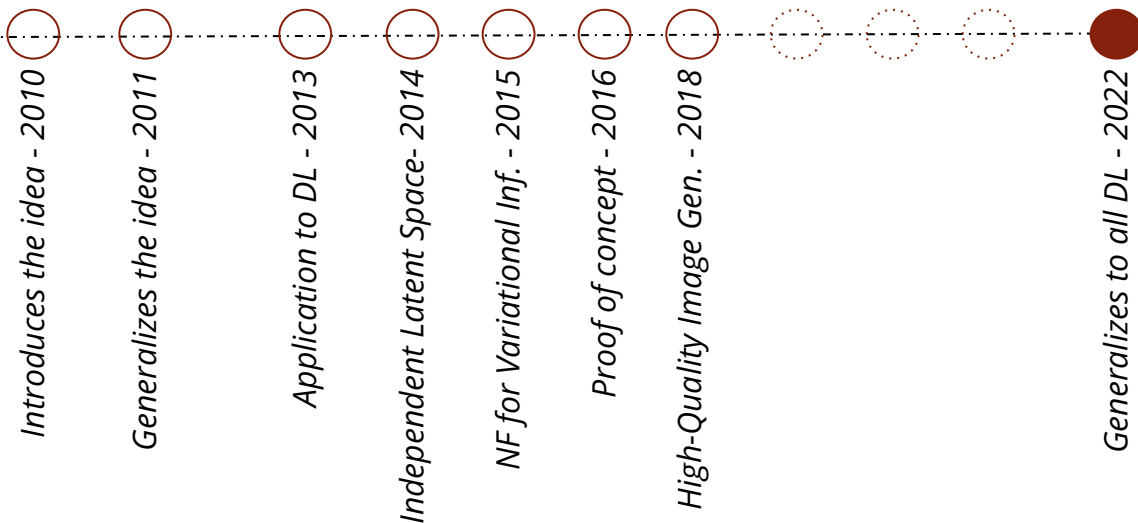
Tobias Golling
University of Geneva
tobias.golling@unige.ch

François Fleuret
University of Geneva
francois.fleuret@unige.ch

The authors propose that certain neural network architectures, which include linear layers, convolutional layers, and invertible activations such as LeakyReLU, can be enriched with a **stochastic inverse pass**. This stochastic inverse pass allows for monitoring the likelihood contribution during the flowification process.

Flow-based GMs Literature

argues that by introducing a stochastic inverse pass and monitoring the likelihood contribution, certain neural network architectures can be considered generalized normalizing flows.



Flowification: Everything is a Normalizing Flow

Bálint Máté*
University of Geneva
balint.mate@unige.ch

Samuel Klein*
University of Geneva
samuel.klein@unige.ch

Tobias Golling
University of Geneva
tobias.golling@unige.ch

François Fleuret
University of Geneva
francois.fleuret@unige.ch

The paper introduces the concept of **flowification**, which refers to the enrichment of specific neural network architectures with the characteristics of normalizing flows through the stochastic inverse pass and likelihood monitoring. The flowification process enables these neural networks to fall under the generalized notion of normalizing flows mentioned earlier.

Normalizing Flows Fundamentals

- Change of Variables

$$p_{\mathbf{X}}(\mathbf{x}) = p_{\mathbf{Z}}(f(\mathbf{x})) \left| \det Df(\mathbf{x}) \right|$$

where $\mathbf{Z} = f(\mathbf{X})$ is an invertible, differentiable function
and $Df(\mathbf{x})$ is the Jacobian of $f(\mathbf{x})$

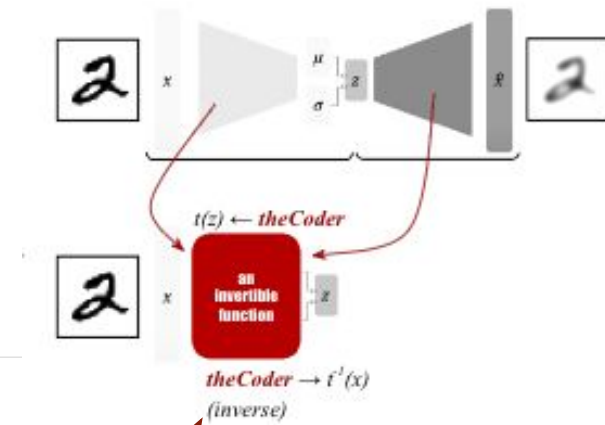
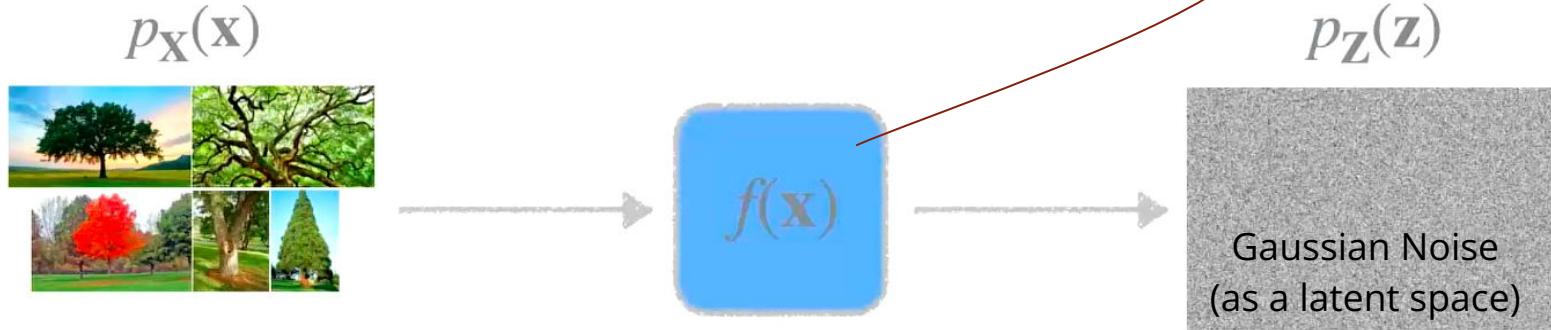
$$\mathbf{J} = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \cdots & \frac{\partial f}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \nabla^T f_1 \\ \vdots \\ \nabla^T f_m \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Normalizing Flows Fundamentals

- Change of Variables

Can represent a given $p_{\mathbf{X}}(\mathbf{x})$ in terms of $p_{\mathbf{Z}}(\mathbf{z})$ and $f(\mathbf{x})$

$$p_{\mathbf{X}}(\mathbf{x}) = p_{\mathbf{Z}}(f(\mathbf{x})) \left| \det Df(\mathbf{x}) \right|$$



Normalizing Flows Fundamentals

- Learn $f(\mathbf{x})$ to transform data distribution $p_{\mathbf{X}}(\mathbf{x})$ into $p_{\mathbf{Z}}(\mathbf{z})$

Two pieces

- **Base Measure:** $p_{\mathbf{Z}}(\mathbf{z})$ - Typically selected as $\mathcal{N}(\mathbf{z} | \mathbf{0}, \mathbf{I})$
- **Flow:** $f(\mathbf{x})$ - Must be invertible and differentiable

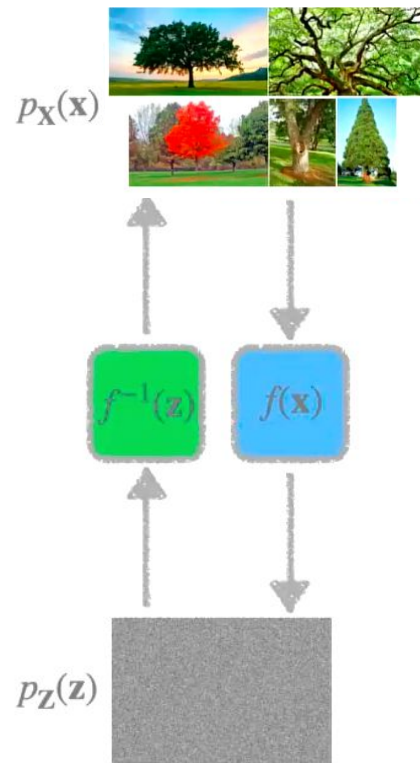
Normalizing Flows Fundamentals

- Density evaluation: Determinant of the Jacobian

$$p_{\mathbf{X}}(\mathbf{x}) = p_{\mathbf{Z}}(f(\mathbf{x})) \left| \det Df(\mathbf{x}) \right|$$

Sampling:

- Sample $\mathbf{z} \sim p_{\mathbf{Z}}(\cdot)$
- Compute $\mathbf{x} = f^{-1}(\mathbf{z})$



How to train

- What is the likelihood here?

$$p_{\mathbf{X}}(\mathbf{x}) = p_{\mathbf{Z}}(f(\mathbf{x})) \left| \det Df(\mathbf{x}) \right|$$

- Then train the maximum (log) likelihood

$$\max_{\theta} \sum_{i=1}^N \log p_{\mathbf{Z}}(f(\mathbf{x}_i | \theta)) + \log \left| \det Df(\mathbf{x}_i | \theta) \right|$$

What are these flows?

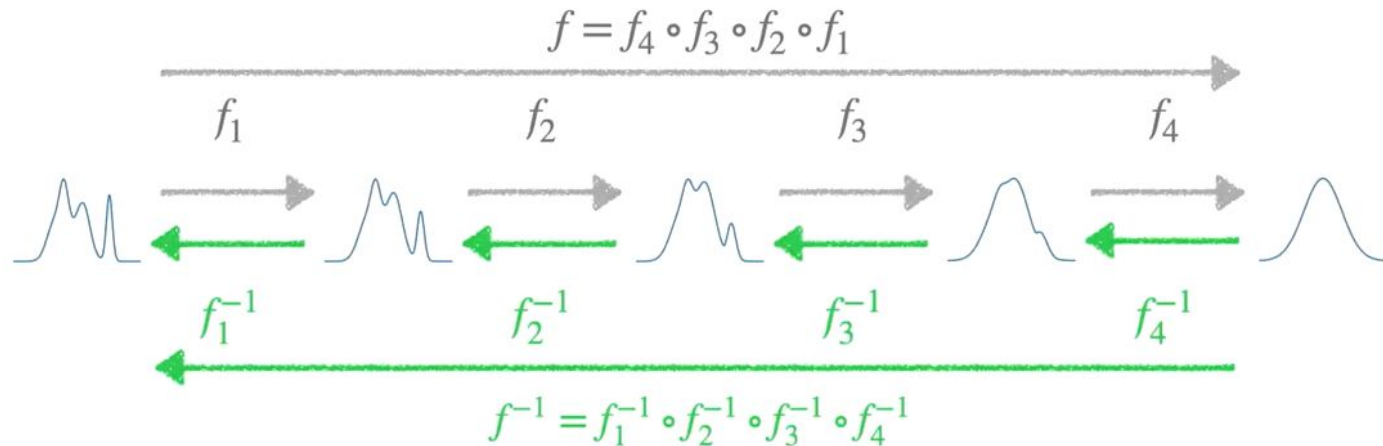
- Ok, great idea. Congrats to whomever came up with it. We love those people. But what is this “flow” practically? Is it a layer, is it a function, is this love that I’m feeling?

A **flow** is a parametric function $f(\mathbf{x})$ which:

- is invertible
- is differentiable
- has an efficiently computable inverse and Jacobian determinant $|\det Df(\mathbf{x})|$

What are these flows?

- Ok, great idea. Congrats to whomever came up with it. We love those people. But what is this “flow” practically? Is it a layer, is it a function?



What are these flows?

- When doing this we will have to calculate a lot of determinants, because of the underlying mathematics:

Determinant:

$$\det Df = \det \prod_{k=1}^K Df_k = \prod_{k=1}^K \det Df_k$$

Likelihood:

$$\max_{\theta} \sum_{i=1}^N \log p_{\mathbf{Z}}(f(\mathbf{x}_i | \theta)) + \sum_{k=1}^K \log |\det Df_k(\mathbf{x}_i | \theta)|$$

**Glow: Generative Flow
with Invertible 1×1 Convolutions**

Diederik P. Kingma*, Prafulla Dhariwal*
OpenAI, San Francisco

Linear Flows

A linear transformation can be a flow if the matrix is invertible

$$f(\mathbf{x}) = \mathbf{Ax} + \mathbf{b}$$

Inverse: $f^{-1}(\mathbf{z}) = \mathbf{A}^{-1}(\mathbf{z} - \mathbf{b})$

Determinant: $\det Df(\mathbf{x}) = \det \mathbf{A}$

Problem:

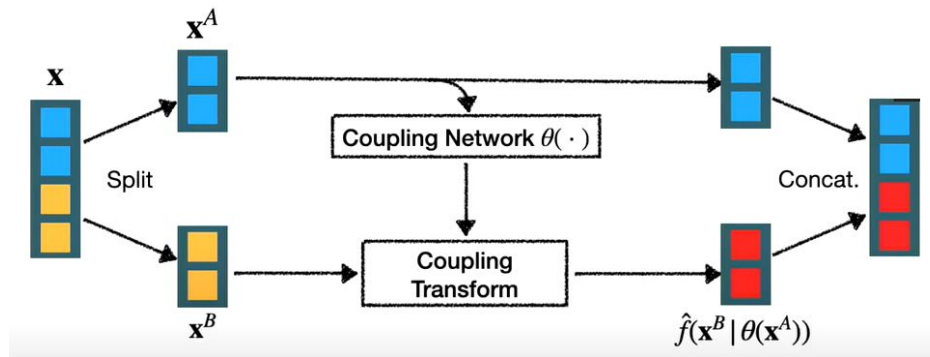
- Inexpressive (linear functions are closed under composition)
- Determinant/inverse could be $O(d^3)$

	Inverse	Determinant
Full	$O(d^3)$	$O(d^3)$
Diagonal	$O(d)$	$O(d)$
Triangular	$O(d^2)$	$O(d)$
Block Diagonal	$O(c^3d)$	$O(c^3d)$
LU Factorized [Kingma and Dhariwal 2018]	$O(d^2)$	$O(d)$
Spatial Convolution [Hoogeboom et al 2018; Karimi et al., 2018]	$O(d \log d)$	$O(d)$
1x1 Convolution [Kingma and Dhariwal 2018]	$O(c^3 + c^2d)$	$O(c^3)$

Coupling Flows

In a coupling flow, the data is split into two parts, often referred to as "channels" or "scales."

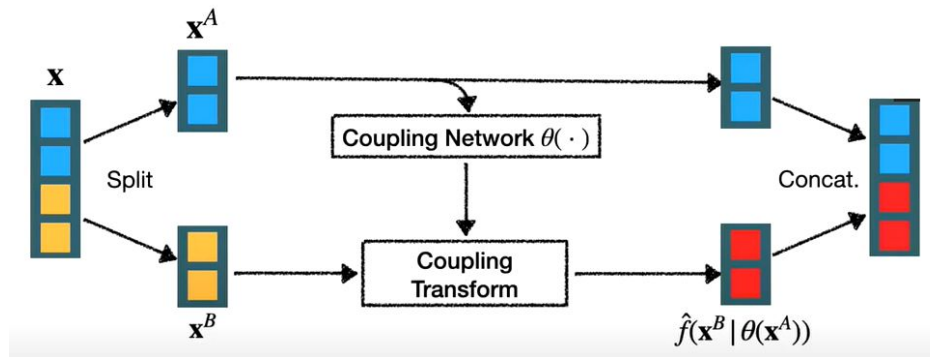
One part remains unchanged while the other part undergoes a learned, invertible transformation.



Coupling Flows

This clever division ensures that the determinant of the Jacobian matrix for the overall transformation is relatively simple to compute, as it is the product of the determinants of the transformed and unchanged parts separately.

This allows for efficient and tractable modeling of complex probability distributions using normalizing flows.



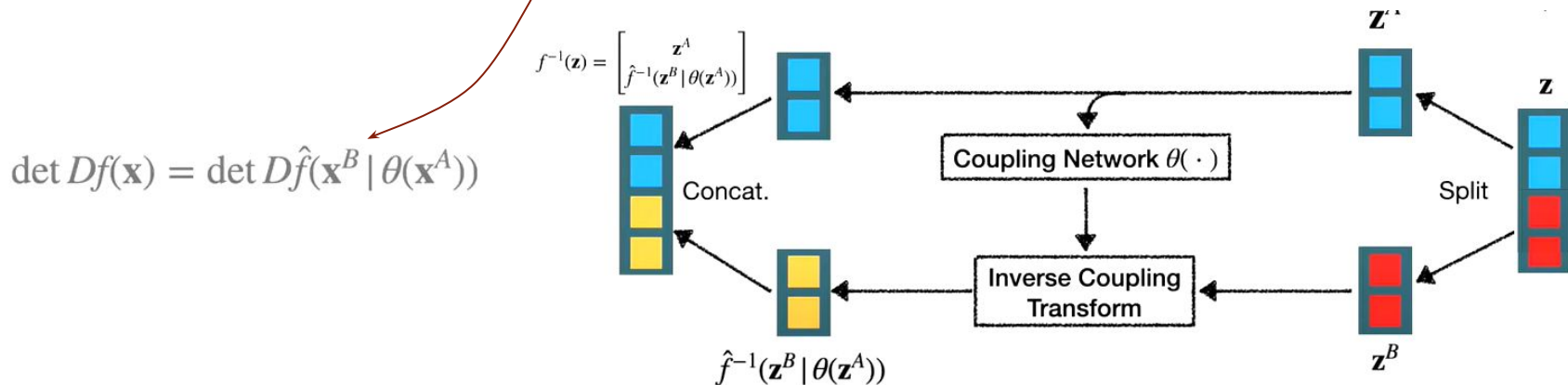
Coupling Flows

$$\mathbf{J} = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x_1} & \cdots & \frac{\partial \mathbf{f}}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \nabla^T f_1 \\ \vdots \\ \nabla^T f_m \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

The reverse is simply the reverse operation in the computational graph.

$$Df(\mathbf{x}) = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \frac{\partial}{\partial \mathbf{x}^A} \hat{f}(\mathbf{x}^B | \theta(\mathbf{x}^A)) & D\hat{f}(\mathbf{x}^B | \theta(\mathbf{x}^A)) \end{bmatrix}$$

Can make $\theta(\mathbf{x}^A)$ arbitrarily complex, e.g., MLP, CNN, etc



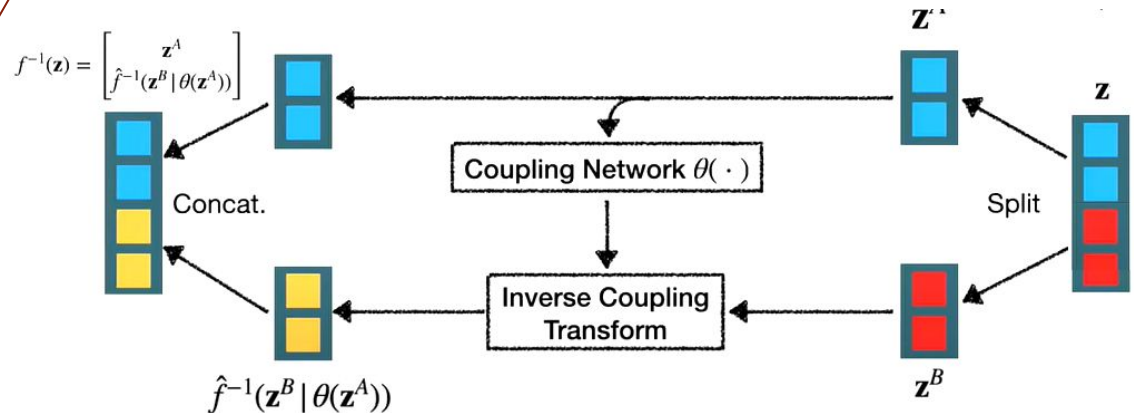
Coupling Flows

The calculation of the Jacobian becomes:

$$Df(\mathbf{x}) = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \frac{\partial}{\partial \mathbf{x}^A} \hat{f}(\mathbf{x}^B | \theta(\mathbf{x}^A)) & D\hat{f}(\mathbf{x}^B | \theta(\mathbf{x}^A)) \end{bmatrix}$$

Can make $\theta(\mathbf{x}^A)$ arbitrarily complex, e.g., MLP, CNN, etc

$$\det Df(\mathbf{x}) = \det D\hat{f}(\mathbf{x}^B | \theta(\mathbf{x}^A))$$

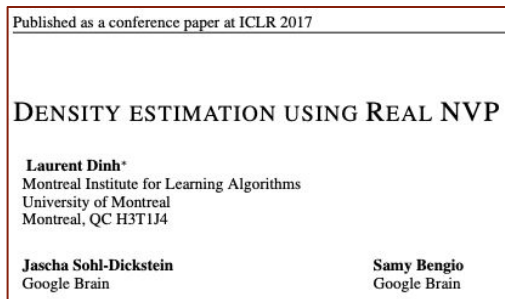




Dinh et al. 2016

Most of the papers in NF literature exploited this idea to different functions (NICE, Flow++,etc)

Let's delve into one of them:



Published as a conference paper at ICLR 2017

DENSITY ESTIMATION USING REAL NVP

Laurent Dinh*
Montreal Institute for Learning Algorithms
University of Montreal
Montreal, QC H3T1J4

Jascha Sohl-Dickstein
Google Brain

Samy Bengio
Google Brain

ABSTRACT

Unsupervised learning of probabilistic models is a central yet challenging problem in machine learning. Specifically, designing models with tractable learning, sampling, inference and evaluation is crucial in solving this task. We extend the space of such models using real-valued non-volume preserving (real NVP) transformations, a set of powerful, stably invertible, and learnable transformations, resulting in an unsupervised learning algorithm with exact log-likelihood computation, exact and efficient sampling, exact and efficient inference of latent variables, and an interpretable latent space. We demonstrate its ability to model natural images on four datasets through sampling, log-likelihood evaluation, and latent variable manipulations.

1 Introduction

Dinh et al. 2016

The paper defines its aim as “unsupervised learning of probabilistic models”, not generative modelling (although it is one of the aims)

“We extend the space probabilistic models using real-valued non-volume preserving (real NVP) transformations, a set of powerful, stably invertible, and learnable transformations, resulting in an unsupervised learning algorithm with exact log-likelihood computation, exact and efficient sampling, exact and efficient inference of latent variables, and an interpretable latent space.”

Published as a conference paper at ICLR 2017

DENSITY ESTIMATION USING REAL NVP

Laurent Dinh*

Montreal Institute for Learning Algorithms
University of Montreal
Montreal, QC H3T1J4

Jascha Sohl-Dickstein
Google Brain

Samy Bengio
Google Brain

ABSTRACT

Unsupervised learning of probabilistic models is a central yet challenging problem in machine learning. Specifically, designing models with tractable learning, sampling, inference and evaluation is crucial in solving this task. We extend the space of such models using real-valued non-volume preserving (real NVP) transformations, a set of powerful, stably invertible, and learnable transformations, resulting in an unsupervised learning algorithm with exact log-likelihood computation, exact and efficient sampling, exact and efficient inference of latent variables, and an interpretable latent space. We demonstrate its ability to model natural images on four datasets through sampling, log-likelihood evaluation, and latent variable manipulations.

1 Introduction



Dinh et al. 2016

The paper defines its aim as “unsupervised learning of probabilistic models”, not generative modelling (although it one of the aims)

“We extend the space probabilistic models using real-valued non-volume preserving (real NVP) transformations, a set of powerful, stably invertible, and learnable transformations, resulting in an unsupervised learning algorithm with exact log-likelihood computation, exact and efficient sampling, exact and efficient inference of latent variables, and an interpretable latent space.”

what is non-volume preserving in this context and more importantly, but in the context of normalizing flows, the determinant in the change of variables formula does guarantee that at every transformation step we make sure that each intermediate probability density function integrates to 1. so does this NVP idea contradict with it? if so, why? is it because in this paper the idea of normalizing flow was not totally matured?

chatPDF:

In the context of normalizing flows, the determinant in the change of variables formula guarantees that each intermediate probability density function integrates to 1, which is important for tractable learning and inference. The non-volume preserving (NVP) transformations used in this paper do not preserve volume, meaning that the determinant of the Jacobian matrix is not necessarily 1. However, the authors show that the NVP transformations are still tractable and expressive, and they propose a new change of variables formula that accounts for the non-volume preserving property. This formula enables exact log-likelihood computation and efficient sampling, while still ensuring that the intermediate probability density functions integrate to 1. Therefore, the NVP idea does not contradict the idea of normalizing flows, but rather extends it to a new class of transformations that are more flexible and powerful.

DENSITY ESTIMATION USING REAL NVP

Laurent Dinh*

Montreal Institute for Learning Algorithms
University of Montreal
Montreal, QC H3T1J4

Jascha Sohl-Dickstein

Google Brain

Samy Bengio

Google Brain

ABSTRACT

Unsupervised learning of probabilistic models is a central yet challenging problem in machine learning. Specifically, designing models with tractable learning, sampling, inference and evaluation is crucial in solving this task. We extend the space of such models using real-valued non-volume preserving (real NVP) transformations, a set of powerful, stably invertible, and learnable transformations, resulting in an unsupervised learning algorithm with exact log-likelihood computation, exact and efficient sampling, exact and efficient inference of latent variables, and an interpretable latent space. We demonstrate its ability to model natural images on four datasets through sampling, log-likelihood evaluation, and latent variable manipulations.

1 Introduction



Dinh et al. 2016

The paper defines its aim as “unsupervised learning of probabilistic models”, not generative modelling (although it one of the aims)

“We extend the space probabilistic models using real-valued non-volume preserving (real NVP) transformations, a set of powerful, stably invertible, and learnable transformations, resulting in an unsupervised learning algorithm with exact log-likelihood computation, exact and efficient sampling, exact and efficient inference of latent variables, and an interpretable latent space.”

chatPDF:

so if it is not volume preserving how do they make sure that the resulting distribution integrates to 1?

Although the NVP transformations used in this paper are not volume preserving, the authors propose a new change of variables formula that accounts for the non-volume preserving property. This formula involves a Jacobian matrix that is not necessarily diagonal, but can still be computed efficiently. The determinant of this Jacobian matrix is not necessarily 1, but the authors show that it can be easily computed and used to adjust the probability density function of the transformed variable. Specifically, the authors use a logit transformation to map the original data to an unbounded space, and then apply a series of NVP transformations to model the density of the transformed variable. The final probability density function is obtained by applying the inverse logit transformation and adjusting for the Jacobian determinant of each NVP transformation. This ensures that the resulting distribution integrates to 1 while

Published as a conference paper at ICLR 2017

DENSITY ESTIMATION USING REAL NVP

Laurent Dinh*

Montreal Institute for Learning Algorithms
University of Montreal
Montreal, QC H3T1J4

Jascha Sohl-Dickstein
Google Brain

Samy Bengio
Google Brain

ABSTRACT

Unsupervised learning of probabilistic models is a central yet challenging problem in machine learning. Specifically, designing models with tractable learning, sampling, inference and evaluation is crucial in solving this task. We extend the space of such models using real-valued non-volume preserving (real NVP) transformations, a set of powerful, stably invertible, and learnable transformations, resulting in an unsupervised learning algorithm with exact log-likelihood computation, exact and efficient sampling, exact and efficient inference of latent variables, and an interpretable latent space. We demonstrate its ability to model natural images on four datasets through sampling, log-likelihood evaluation, and latent variable manipulations.

1 Introduction



Dinh et al. 2016

Probabilistic Undirected Graphs: The authors mention that one class of generative models used for maximum likelihood training is based on probabilistic undirected graphs. These models are also known as energy-based models.

Conditional Independence Property: RBMs and DBMs are structured in a way that latent variables are conditionally independent given the observed data. This means that the latent variables (i.e. hidden states) can be updated efficiently during training because they don't have complex dependencies among themselves, which is often the case in other models like fully connected neural networks.

While this can lead to more efficient training and inference for these models, it also imposes a constraint on the types of dependencies they can capture. In other words, they may not be as flexible in modeling complex dependencies as Normalizing Flows.

2 Related work

Substantial work on probabilistic generative models has focused on training models using maximum likelihood. One class of maximum likelihood models are those described by *probabilistic undirected graphs*, such as *Restricted Boltzmann Machines* [58] and *Deep Boltzmann Machines* [53]. These models are trained by taking advantage of the conditional independence property of their bipartite structure to allow efficient exact or approximate posterior inference on latent variables. However, because of the intractability of the associated marginal distribution over latent variables, their training, evaluation, and sampling procedures necessitate the use of approximations like *Mean Field inference* and *Markov Chain Monte Carlo*, whose convergence time for such complex models remains undetermined, often resulting in generation of highly correlated samples. Furthermore, these approximations can often hinder their performance [7].

Directed graphical models are instead defined in terms of an *ancestral sampling procedure*, which is appealing both for its conceptual and computational simplicity. They lack, however, the conditional independence structure of undirected models, making exact and approximate posterior inference on latent variables cumbersome [56]. Recent advances in *stochastic variational inference* [27] and *amortized inference* [13, 43, 35, 49], allowed efficient approximate inference and learning of deep directed graphical models by maximizing a variational lower bound on the log-likelihood [45]. In particular, the *variational autoencoder algorithm* [35, 49] simultaneously learns a *generative network*, that maps gaussian latent variables z to samples x , and a *matched approximate inference network* that maps samples x to a semantically meaningful latent representation z , by exploiting the *reparametrization trick* [68]. Its success in leveraging recent advances in backpropagation [51, 39] in deep neural networks resulted in its adoption for several applications ranging from speech synthesis [12] to language modeling [8]. Still, the approximation in the inference process limits its ability to learn high dimensional deep representations, motivating recent work in improving approximate inference [42, 48, 55, 63, 10, 59, 34].

Such approximations can be avoided altogether by abstaining from using latent variables. *Autoregressive models* [18, 6, 37, 20] can implement this strategy while typically retaining a great deal of flexibility. This class of algorithms tractably models the joint distribution by decomposing it into a product of conditionals using the *probability chain rule* according to a fixed ordering over dimensions, simplifying log-likelihood evaluation and sampling. Recent work in this line of research has taken advantage of recent advances in *recurrent networks* [51], in particular *long-short term memory* [26], and *residual networks* [25, 24] in order to learn state-of-the-art generative image models [61, 46] and language models [32]. The ordering of the dimensions, although often arbitrary, can be critical to the training of the model [66]. The sequential nature of this model limits its computational efficiency. For example, its sampling procedure is sequential and non-parallelizable, which can become cumbersome in applications like speech and music synthesis, or real-time rendering. Additionally, there is no natural latent representation associated with autoregressive models, and they have not yet been shown to be useful for semi-supervised learning.

Dinh et al. 2016

Ancestral sampling is a process where you sample from the model starting from the top (observed variables) and moving down to the latent variables.

"directed" and "undirected" refer to how variables and their probabilistic dependencies are represented in graphical models, with directed models using arrows to indicate causal or probabilistic relationships, and undirected models using undirected edges to represent symmetric relationships.

Directed models (VAEs) lack the conditional independence structure seen in undirected models. This means that exact and approximate inference on latent variables in directed models can be more challenging.

Amortized inference: the fact that VAEs are not trained on exact likelihood (instead trained on ELBO)

2 Related work

Substantial work on probabilistic generative models has focused on training models using maximum likelihood. One class of maximum likelihood models are those described by *probabilistic undirected graphs*, such as *Restricted Boltzmann Machines* [58] and *Deep Boltzmann Machines* [53]. These models are trained by taking advantage of the conditional independence property of their bipartite structure to allow efficient exact or approximate posterior inference on latent variables. However, because of the intractability of the associated marginal distribution over latent variables, their training, evaluation, and sampling procedures necessitate the use of approximations like *Mean Field inference* and *Markov Chain Monte Carlo*, whose convergence time for such complex models remains undetermined, often resulting in generation of highly correlated samples. Furthermore, these approximations can often hinder their performance [7].

Directed graphical models are instead defined in terms of an *ancestral sampling procedure*, which is appealing both for its conceptual and computational simplicity. They lack, however, the conditional independence structure of undirected models, making exact and approximate posterior inference on latent variables cumbersome [56]. Recent advances in *stochastic variational inference* [27] and *amortized inference* [13, 43, 35, 49], allowed efficient approximate inference and learning of deep directed graphical models by maximizing a variational lower bound on the log-likelihood [45]. In particular, the *variational autoencoder algorithm* [35, 49] simultaneously learns a *generative network*, that maps gaussian latent variables z to samples x , and a matched *approximate inference network* that maps samples x to a semantically meaningful latent representation z , by exploiting the *reparametrization trick* [68]. Its success in leveraging recent advances in backpropagation [51, 39] in deep neural networks resulted in its adoption for several applications ranging from speech synthesis [12] to language modeling [8]. Still, the approximation in the inference process limits its ability to learn high dimensional deep representations, motivating recent work in improving approximate inference [42, 48, 55, 63, 10, 59, 34].

Such approximations can be avoided altogether by abstaining from using latent variables. *Auto-regressive models* [18, 6, 37, 20] can implement this strategy while typically retaining a great deal of flexibility. This class of algorithms tractably models the joint distribution by decomposing it into a product of conditionals using the *probability chain rule* according to a fixed ordering over dimensions, simplifying log-likelihood evaluation and sampling. Recent work in this line of research has taken advantage of recent advances in *recurrent networks* [51], in particular *long-short term memory* [26], and *residual networks* [25, 24] in order to learn state-of-the-art generative image models [61, 46] and language models [32]. The ordering of the dimensions, although often arbitrary, can be critical to the training of the model [66]. The sequential nature of this model limits its computational efficiency. For example, its sampling procedure is sequential and non-parallelizable, which can become cumbersome in applications like speech and music synthesis, or real-time rendering. Additionally, there is no natural latent representation associated with autoregressive models, and they have not yet been shown to be useful for semi-supervised learning.

Dinh et al. 2016

In semi-supervised learning, the presence of latent variables can be highly beneficial. Latent variables serve as a compact and meaningful representation of the data. For example, in VAEs, the latent space represents features or characteristics of the data.

By learning a good latent representation, a model can generalize better from labeled data to unlabeled data. The model can leverage the latent space to identify common patterns and shared information between labeled and unlabeled samples.

2 Related work

Substantial work on probabilistic generative models has focused on training models using maximum likelihood. One class of maximum likelihood models are those described by *probabilistic undirected graphs*, such as *Restricted Boltzmann Machines* [58] and *Deep Boltzmann Machines* [53]. These models are trained by taking advantage of the conditional independence property of their bipartite structure to allow efficient exact or approximate posterior inference on latent variables. However, because of the intractability of the associated marginal distribution over latent variables, their training, evaluation, and sampling procedures necessitate the use of approximations like *Mean Field inference* and *Markov Chain Monte Carlo*, whose convergence time for such complex models remains undetermined, often resulting in generation of highly correlated samples. Furthermore, these approximations can often hinder their performance [7].

Directed graphical models are instead defined in terms of an *ancestral sampling procedure*, which is appealing both for its conceptual and computational simplicity. They lack, however, the conditional independence structure of undirected models, making exact and approximate posterior inference on latent variables cumbersome [56]. Recent advances in *stochastic variational inference* [27] and *amortized inference* [13, 43, 35, 49], allowed efficient approximate inference and learning of deep directed graphical models by maximizing a variational lower bound on the log-likelihood [45]. In particular, the *variational autoencoder algorithm* [35, 49] simultaneously learns a *generative network*, that maps gaussian latent variables z to samples x , and a *matched approximate inference network* that maps samples x to a semantically meaningful latent representation z , by exploiting the *reparametrization trick* [68]. Its success in leveraging recent advances in backpropagation [51, 39] in deep neural networks resulted in its adoption for several applications ranging from speech synthesis [12] to language modeling [8]. Still, the approximation in the inference process limits its ability to learn high dimensional deep representations, motivating recent work in improving approximate inference [42, 48, 55, 63, 10, 59, 34].

Such approximations can be avoided altogether by abstaining from using latent variables. *Autoregressive models* [18, 6, 37, 20] can implement this strategy while typically retaining a great deal of flexibility. This class of algorithms tractably models the joint distribution by decomposing it into a product of conditionals using the *probability chain rule* according to a fixed ordering over dimensions, simplifying log-likelihood evaluation and sampling. Recent work in this line of research has taken advantage of recent advances in *recurrent networks* [51], in particular *long-short term memory* [26], and *residual networks* [25, 24] in order to learn state-of-the-art generative image models [61, 46] and language models [32]. The ordering of the dimensions, although often arbitrary, can be critical to the training of the model [66]. The sequential nature of this model limits its computational efficiency. For example, its sampling procedure is sequential and non-parallelizable, which can become cumbersome in applications like speech and music synthesis, or real-time rendering. Additionally, there is no natural latent representation associated with autoregressive models, and they have not yet been shown to be useful for semi-supervised learning.



Dinh et al. 2016

The theory is based on the change of variable (for distributions) formula and coupling. The code exists:

<https://paperswithcode.com/method/realnvp>

real-nvp / models / real_nvp / real_nvp.py

```

chrishute Remove bn.

Code Blame 144 lines (115 loc) · 5.78 KB

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 from models.real_nvp.coupling_layer import CouplingLayer, MaskType
6 from util import squeeze_2x2
7
8
9 class RealNVP(nn.Module):
10     """RealNVP Model
11
12     Based on the paper:
13     "Density estimation using Real NVP"
14     by Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio
15     (https://arxiv.org/abs/1605.08803).
16
17     """

```

3.1 Change of variable formula

Given an observed data variable $x \in X$, a simple prior probability distribution p_Z on a latent variable $z \in Z$, and a bijection $f : X \rightarrow Z$ (with $g = f^{-1}$), the change of variable formula defines a model distribution on X by

$$p_X(x) = p_Z(f(x)) \left| \det \left(\frac{\partial f(x)}{\partial x^T} \right) \right| \quad (2)$$

$$\log(p_X(x)) = \log(p_Z(f(x))) + \log \left(\left| \det \left(\frac{\partial f(x)}{\partial x^T} \right) \right| \right), \quad (3)$$

where $\frac{\partial f(x)}{\partial x^T}$ is the Jacobian of f at x .

Exact samples from the resulting distribution can be generated by using the inverse transform sampling rule [16]. A sample $z \sim p_Z$ is drawn in the latent space, and its inverse image $x = f^{-1}(z) = g(z)$ generates a sample in the original space. Computing the density on a point x is accomplished by computing the density of its image $f(x)$ and multiplying by the associated Jacobian determinant $\det \left(\frac{\partial f(x)}{\partial x^T} \right)$. See also Figure 1. Exact and efficient inference enables the accurate and fast evaluation of the model.

3

Published as a conference paper at ICLR 2017

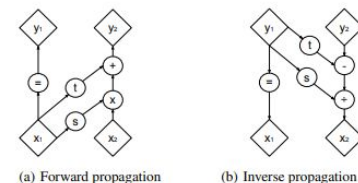


Figure 2: Computational graphs for forward and inverse propagation. A coupling layer applies a simple invertible transformation consisting of scaling followed by addition of a constant offset to one part x_2 of the input vector conditioned on the remaining part of the input vector x_1 . Because of its simple nature, this transformation is both easily invertible and possesses a tractable determinant. However, the conditional nature of this transformation, captured by the functions s and t , significantly increase the flexibility of this otherwise weak function. The forward and inverse propagation operations have identical computational cost.

What will we do next week?

- Next week (Nov 7): Diffusion models
- The other week (Nov 14): Presentation by Barış Yılmaz “TimesNet-Gen”
- The other, other week (Nov 21): the Midterm

Additional Reading & References

- [Flow-based Deep Generative Models | Lil'Log](#)
- <https://www.youtube.com/watch?v=bXp8fk4MRXQ>
- <https://www.youtube.com/watch?v=8XufsgG066A&t=2868s>
- <https://onlinelibrary.wiley.com/doi/epdf/10.1002/cpa.21423>
- <https://math.nyu.edu/~tabak/publications/CMSV8-1-10.pdf>
- <https://paperswithcode.com/method/realnvp>